From the output, it can be seen that the debug and info messages are not being logged, as the default severity is set to 'warning'. So messages will be printed for the warning level and above.

Let us now see how the severity levels can be changed by changing the configurations. We will also look at how to change the format of the output, later on.

Parameter	Description
level	Root logger will be set to the specified severity level
filename	Name of the file logs should be stored
filemode	Mode in which the log file should be opened; default is 'a' append mode
format	Pattern for the log message to be written
datefmt	Format in which date should be written

Table 2: Commonly used log parameters

Several configurations can be done.

Basic configuration

We can use the method given below to configure the logging:

```
import logging
logging.basicConfig(level=logging.DEBUG, filename='/project/
log/app.log', filemode='w', format='%(asctime)s - %(message)
s', datefmt='%d-%b-%y %H:%M:%S')

logging.debug('This is a debug message')
logging.info('This is an info message')
logging.warning('This is a warning message')
logging.error('This is an error message')
logging.critical('This is a critical message')
```

The output (from *app.log*) is:

```
27-Nov-21 07:28:09 - This is a debug message
27-Nov-21 07:28:09 - This is an info message
27-Nov-21 07:28:09 - This is a warning message
27-Nov-21 07:28:09 - This is an error message
27-Nov-21 07:28:09 - This is a critical message
```

Through this basic configuration we observe that:

- Debug and info messages are logged, as the logging level is now 'debug' and above.
- Log contents are not displayed in *stdout;* rather, they are saved in the file called *app.log.*
- Message format is different, because it works as per the format given in the format parameter.
- The date format is also different.

File configuration

Before we get to know the file configuration, let's look at how logging works.

Logging flow: So far, we have seen the default logger named root, which is used by the logging module whenever its functions are called directly, like *logging.debug()*. But it is preferable to define your own logger.

The logging module follows the modular approach that splits functionalities into components.

1. **Logger:** This is exposed to application code for direct usage, and the logging is performed by calling methods on instances of the Logger class. This class performs three duties.
   - Exposes several methods to application code so that applications can log messages at runtime. A good convention is to use the module name to name the loggers.
     - `logger = logging.getLogger(__name__)`
   - Logger objects determine which log messages to act upon based upon the severity or filter objects.
     - `Logger.setLevel()`
     - `Logger.addFilter()and Logger.removeFilter()`
   - Logger objects pass along relevant log messages to all interested log handlers.
     - `Logger.addFilter()and Logger.removeFilter()`
2. **LogRecord:** Loggers automatically create LogRecord objects that have all the information related to the event being logged, like the name of the logger, function, line number, message, and more.
3. **Filter:** This provides a finer grained facility for determining which log records to output. We will later see an example that uses filters.
4. **Handler:** This sends the LogRecords (created by loggers) to the appropriate destination. For example, it could be file, stdout, http, etc. For each destination, handler has a subclass such as file: FileHandler, stdout: StreamHandler, http: HTTPHandler. Logger objects can add zero or more handler objects to themselves with an *addHandler()* method.
5. **Formatter:** This specifies the format of LogRecord by mentioning it in the list of attributes that need to be logged in order to be written in the final output.

We can configure logging in three ways:

1. By creating loggers, handlers, and formatters explicitly using Python code that calls the configuration methods. In the example given below, though a logger is created with the same module name, the console handler is created